JavaScript is disabled on your browser.

Class SecurityManager

java.lang.Object
    java.lang.SecurityManager

Direct Known Subclasses:

RMISecurityManager

```
public class SecurityManager
extends Object
```

The security manager is a class that allows applications to implement a security policy. It allows an application to determine, before performing a possibly unsafe or sensitive operation, what the operation is and whether it is being attempted in a security context that allows the operation to be performed. The application can allow or disallow the operation.

The `SecurityManager` class contains many methods with names that begin with the word `check`. These methods are called by various methods in the Java libraries before those methods perform certain potentially sensitive operations. The invocation of such a `check` method typically looks like this:

```
SecurityManager security = System.getSecurityManager();
if (security != null) {
    security.checkXXX(argument,  . . . );
}
```

The security manager is thereby given an opportunity to prevent completion of the operation by throwing an exception. A security manager routine simply returns if the operation is permitted, but throws a `SecurityException` if the operation is not permitted. The only exception to this convention is `checkTopLevelWindow`, which returns a `boolean` value.

The current security manager is set by the `setSecurityManager` method in class `System`. The current security manager is obtained by the `getSecurityManager` method.

The special method `checkPermission(java.security.Permission)` determines whether an access request indicated by a specified permission should be granted or denied. The default implementation calls

```
AccessController.checkPermission(perm);
```

If a requested access is allowed, `checkPermission` returns quietly. If denied, a `SecurityException` is thrown.

As of Java 2 SDK v1.2, the default implementation of each of the other `check` methods in `SecurityManager` is to call the `SecurityManager checkPermission` method to determine if the calling thread has permission to perform the requested operation.

Note that the `checkPermission` method with just a single permission argument always performs security checks within the context of the currently executing thread. Sometimes a security check that should be made within a given context will actually need to be done from within a *different* context (for example, from within a worker thread). The `getSecurityContext` method and the `checkPermission` method that includes a context argument are provided for this situation. The `getSecurityContext` method returns a "snapshot" of the current calling context. (The default implementation returns an AccessControlContext object.) A sample call is the following:

```
Object context = null;
SecurityManager sm = System.getSecurityManager();
if (sm != null) context = sm.getSecurityContext();
```

The `checkPermission` method that takes a context object in addition to a permission makes access decisions based on that context, rather than on that of the current execution thread. Code within a different context can thus call that method, passing the permission and the previously-saved context object. A sample call, using the SecurityManager `sm` obtained as in the previous example, is the following:

```
if (sm != null) sm.checkPermission(permission, context);
```

Permissions fall into these categories: File, Socket, Net, Security, Runtime, Property, AWT, Reflect, and Serializable. The classes managing these various permission categories are `java.io.FilePermission`, `java.net.SocketPermission`, `java.net.NetPermission`, `java.security.SecurityPermission`, `java.lang.RuntimePermission`, `java.util.PropertyPermission`, `java.awt.AWTPermission`, `java.lang.reflect.ReflectPermission`, and `java.io.SerializablePermission`.

All but the first two (FilePermission and SocketPermission) are subclasses of `java.security.BasicPermission`, which itself is an abstract subclass of the top-level class for permissions, which is `java.security.Permission`. BasicPermission defines the functionality needed for all permissions that contain a name that follows the hierarchical property naming convention (for example, "exitVM", "setFactory", "queuePrintJob", etc). An asterisk may appear at the end of the name, following a ".", or by itself, to signify a wildcard match. For example: "a.*" or "*" is valid, "*a" or "a*b" is not valid.

FilePermission and SocketPermission are subclasses of the top-level class for permissions (`java.security.Permission`). Classes like these that have a more complicated name syntax than that used by BasicPermission subclass directly from Permission rather than from BasicPermission. For example, for a `java.io.FilePermission` object, the permission name is the path name of a file (or directory).

Some of the permission classes have an "actions" list that tells the actions that are permitted for the object. For example, for a `java.io.FilePermission` object, the actions list (such as "read, write") specifies which actions are granted for the specified file (or for files in the specified directory).

Other permission classes are for "named" permissions - ones that contain a name but no actions list; you either have the named permission or you don't.

Note: There is also a `java.security.AllPermission` permission that implies all permissions. It exists to simplify the work of system administrators who might need to perform multiple tasks that require all (or numerous) permissions.

See Permissions in the JDK for permission-related information. This document includes, for example, a table listing the various SecurityManager `check` methods and the permission(s) the default implementation of each such method requires. It also contains a table of all the version 1.2 methods that require permissions, and for each such method tells which permission it requires.

For more information about `SecurityManager` changes made in the JDK and advice regarding porting of 1.1-style security managers, see the security documentation.

Since:

`JDK1.0`

See Also:

`ClassLoader`, `SecurityException`, `checkTopLevelWindow`, `getSecurityManager`, `setSecurityManager`, `AccessController`, `AccessControlContext`, `AccessControlException`, `Permission`, `BasicPermission`, `FilePermission`, `SocketPermission`, `PropertyPermission`, `RuntimePermission`, `AWTPermission`, `Policy`, `SecurityPermission`, `ProtectionDomain`

Field Summary

Fields

Modifier and Type	Field	Description
protected boolean	inCheck	**Deprecated.** This type of security checking is not recommended. It is recommended that the checkPermission call be used instead.

Constructor Summary

Constructors

Constructor	Description
SecurityManager()	Constructs a new SecurityManager.

Method Summary

Deprecated Methods

Modifier and Type	Method	Description
void	checkAccept(String host, int port)	Throws a SecurityExcept thread is not permitted to connection from the specif number.
void	checkAccess(Thread t)	Throws a SecurityExcept thread is not allowed to m argument.
void	checkAccess(ThreadGroup g)	Throws a SecurityExcept thread is not allowed to m argument.
void	checkAwtEventQueueAccess()	**Deprecated.** The dependency on AWTPe impediment to future mod platform. Users of this me invoke checkPermission(java.s

		directly. This method will release to check the permi java.security.AllPermi
void	**checkConnect**(**String** host, int port)	Throws a SecurityExcept thread is not allowed to op to the specified host and p
void	**checkConnect**(**String** host, int port, **Object** context)	Throws a SecurityExcept security context is not allo connection to the specifie
void	**checkCreateClassLoader**()	Throws a SecurityExcept thread is not allowed to cr loader.
void	**checkDelete**(**String** file)	Throws a SecurityExcept thread is not allowed to de
void	**checkExec**(**String** cmd)	Throws a SecurityExcept thread is not allowed to cr
void	**checkExit**(int status)	Throws a SecurityExcept thread is not allowed to ca Machine to halt with the s
void	**checkLink**(**String** lib)	Throws a SecurityExcept thread is not allowed to dy code specified by the strin
void	**checkListen**(int port)	Throws a SecurityExcept thread is not allowed to wa request on the specified lo
void	**checkMemberAccess**(**Class**<?> clazz, int which)	**Deprecated.** This method relies on the depth of 4 which is error-p enforced by the runtime. U should instead invoke **checkPermission(java.s** directly. This method will release to check the permi java.security.AllPermi
void	**checkMulticast**(**InetAddress** maddr)	Throws a SecurityExcept thread is not allowed to us receive) IP multicast.
void	**checkMulticast**(**InetAddress** maddr, byte ttl)	**Deprecated.**

		Use #checkPermission(jav instead
void	**checkPackageAccess**(**String** pkg)	Throws a SecurityExcept thread is not allowed to ac specified by the argument.
void	**checkPackageDefinition**(**String** pkg)	Throws a SecurityExcept thread is not allowed to de package specified by the a
void	**checkPermission**(**Permission** perm)	Throws a SecurityExcept access, specified by the gi permitted based on the se in effect.
void	**checkPermission**(**Permission** perm, **Object** context)	Throws a SecurityExcept security context is denied specified by the given perr
void	**checkPrintJobAccess**()	Throws a SecurityExcept thread is not allowed to in request.
void	**checkPropertiesAccess**()	Throws a SecurityExcept thread is not allowed to ac system properties.
void	**checkPropertyAccess**(**String** key)	Throws a SecurityExcept thread is not allowed to ac property with the specifie
void	**checkRead**(**FileDescriptor** fd)	Throws a SecurityExcept thread is not allowed to re file descriptor.
void	**checkRead**(**String** file)	Throws a SecurityExcept thread is not allowed to re the string argument.
void	**checkRead**(**String** file, **Object** context)	Throws a SecurityExcept security context is not allo specified by the string arg
void	**checkSecurityAccess**(**String** target)	Determines whether the p specified permission targe granted or denied.
void	**checkSetFactory**()	Throws a SecurityExcept thread is not allowed to se

		used by ServerSocket or S handler factory used by UR
void	checkSystemClipboardAccess()	**Deprecated.** The dependency on AWTPe impediment to future mod platform. Users of this me invoke checkPermission(java.s directly. This method will release to check the permi java.security.AllPermis
boolean	checkTopLevelWindow(Object window)	**Deprecated.** The dependency on AWTPe impediment to future mod platform. Users of this me invoke checkPermission(java.s directly. This method will release to check the permi java.security.AllPermis
void	checkWrite(FileDescriptor fd)	Throws a SecurityExcept thread is not allowed to wr descriptor.
void	checkWrite(String file)	Throws a SecurityExcept thread is not allowed to wr by the string argument.
protected int	classDepth(String name)	**Deprecated.** This type of security check recommended. It is recom checkPermission call be u
protected int	classLoaderDepth()	**Deprecated.** This type of security check recommended. It is recom checkPermission call be u
protected ClassLoader	currentClassLoader()	**Deprecated.** This type of security check recommended. It is recom checkPermission call be u
protected Class<?>	currentLoadedClass()	**Deprecated.**

		This type of security check recommended. It is recom checkPermission call be u
protected Class[]	getClassContext()	Returns the current execu of classes.
boolean	getInCheck()	**Deprecated.** This type of security check recommended. It is recom checkPermission call be u
Object	getSecurityContext()	Creates an object that enc execution environment.
ThreadGroup	getThreadGroup()	Returns the thread group instantiate any new thread time this is being called.
protected boolean	inClass(String name)	**Deprecated.** This type of security check recommended. It is recom checkPermission call be u
protected boolean	inClassLoader()	**Deprecated.** This type of security check recommended. It is recom checkPermission call be u

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

inCheck

```
@Deprecated
protected boolean inCheck
```

Deprecated. *This type of security checking is not recommended. It is recommended that the checkPermission call be used instead.*

This field is `true` if there is a security check in progress; `false` otherwise.

Constructor Detail

SecurityManager

```
public SecurityManager()
```

Constructs a new `SecurityManager`.

If there is a security manager already installed, this method first calls the security manager's `checkPermission` method with the `RuntimePermission("createSecurityManager")` permission to ensure the calling thread has permission to create a new security manager. This may result in throwing a `SecurityException`.

Throws:

`SecurityException` - if a security manager already exists and its `checkPermission` method doesn't allow creation of a new security manager.

See Also:

`System.getSecurityManager()`, `checkPermission`, `RuntimePermission`

Method Detail

getInCheck

```
@Deprecated
public boolean getInCheck()
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Tests if there is a security check in progress.

Returns:

the value of the inCheck field. This field should contain true if a security check is in progress, false otherwise.

See Also:

`inCheck`

getClassContext

```
protected Class[] getClassContext()
```

Returns the current execution stack as an array of classes.

The length of the array is the number of methods on the execution stack. The element at index `0` is the class of the currently executing method, the element at index `1` is the class of that method's caller, and so on.

Returns:

`the execution stack.`

currentClassLoader

```
@Deprecated
protected ClassLoader currentClassLoader()
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Returns the class loader of the most recently executing method from a class defined using a non-system class loader. A non-system class loader is defined as being a class loader that is not equal to the system class loader (as returned by `ClassLoader.getSystemClassLoader()`) or one of its ancestors.

This method will return `null` in the following three cases:

1. All methods on the execution stack are from classes defined using the system class loader or one of its ancestors.
2. All methods on the execution stack up to the first "privileged" caller (see `AccessController.doPrivileged(java.security.PrivilegedAction<T>)`) are from classes defined using the system class loader or one of its ancestors.
3. A call to `checkPermission` with `java.security.AllPermission` does not result in a SecurityException.

Returns:

`the class loader of the most recent occurrence on the stack of a method from a class defined using a non-system class loader.`

See Also:

`getSystemClassLoader`, `checkPermission`

currentLoadedClass

```
@Deprecated
protected Class<?> currentLoadedClass()
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Returns the class of the most recently executing method from a class defined using a non-system class loader. A non-system class loader is defined as

being a class loader that is not equal to the system class loader (as returned by `ClassLoader.getSystemClassLoader()`) or one of its ancestors.

This method will return `null` in the following three cases:

1. All methods on the execution stack are from classes defined using the system class loader or one of its ancestors.
2. All methods on the execution stack up to the first "privileged" caller (see `AccessController.doPrivileged(java.security.PrivilegedAction<T>)`) are from classes defined using the system class loader or one of its ancestors.
3. A call to `checkPermission` with `java.security.AllPermission` does not result in a SecurityException.

Returns:

the class of the most recent occurrence on the stack of a method from a class defined using a non-system class loader.

See Also:

`getSystemClassLoader`, `checkPermission`

classDepth

```
@Deprecated
protected int classDepth(String name)
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Returns the stack depth of the specified class.

Parameters:

name - the fully qualified name of the class to search for.

Returns:

the depth on the stack frame of the first occurrence of a method from a class with the specified name; -1 if such a frame cannot be found.

classLoaderDepth

```
@Deprecated
protected int classLoaderDepth()
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Returns the stack depth of the most recently executing method from a class defined using a non-system class loader. A non-system class loader is defined

as being a class loader that is not equal to the system class loader (as returned by `ClassLoader.getSystemClassLoader()`) or one of its ancestors.

This method will return -1 in the following three cases:

1. All methods on the execution stack are from classes defined using the system class loader or one of its ancestors.
2. All methods on the execution stack up to the first "privileged" caller (see `AccessController.doPrivileged(java.security.PrivilegedAction<T>)`) are from classes defined using the system class loader or one of its ancestors.
3. A call to `checkPermission` with `java.security.AllPermission` does not result in a SecurityException.

Returns:

`the depth on the stack frame of the most recent occurrence of a method from a class defined using a non-system class loader.`

See Also:

`getSystemClassLoader`, `checkPermission`

inClass

```
@Deprecated
protected boolean inClass(String name)
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Tests if a method from a class with the specified name is on the execution stack.

Parameters:

`name - the fully qualified name of the class.`

Returns:

`true if a method from a class with the specified name is on the execution stack; false otherwise.`

inClassLoader

```
@Deprecated
protected boolean inClassLoader()
```

Deprecated. *This type of security checking is not recommended. It is recommended that the `checkPermission` call be used instead.*

Basically, tests if a method from a class defined using a class loader is on the execution stack.

Returns:

`true if a call to currentClassLoader has a non-null return value.`

See Also:

`currentClassLoader`

getSecurityContext

```
public Object getSecurityContext()
```

Creates an object that encapsulates the current execution environment. The result of this method is used, for example, by the three-argument `checkConnect` method and by the two-argument `checkRead` method. These methods are needed because a trusted method may be called on to read a file or open a socket on behalf of another method. The trusted method needs to determine if the other (possibly untrusted) method would be allowed to perform the operation on its own.

The default implementation of this method is to return an `AccessControlContext` object.

Returns:

`an implementation-dependent object that encapsulates sufficient information about the current execution environment to perform some security checks later.`

See Also:

`checkConnect`, `checkRead`, `AccessControlContext`

checkPermission

```
public void checkPermission(Permission perm)
```

Throws a `SecurityException` if the requested access, specified by the given permission, is not permitted based on the security policy currently in effect.

This method calls `AccessController.checkPermission` with the given permission.

Parameters:

`perm - the requested permission.`

Throws:

`SecurityException - if access is not permitted based on the current security policy.`

`NullPointerException - if the permission argument is null.`

Since:

1.2

checkPermission

```
public void checkPermission(Permission perm,
                            Object context)
```

Throws a `SecurityException` if the specified security context is denied access to the resource specified by the given permission. The context must be a security context returned by a previous call to `getSecurityContext` and the access control decision is based upon the configured security policy for that security context.

If `context` is an instance of `AccessControlContext` then the `AccessControlContext.checkPermission` method is invoked with the specified permission.

If `context` is not an instance of `AccessControlContext` then a `SecurityException` is thrown.

Parameters:

`perm - the specified permission`

`context - a system-dependent security context.`

Throws:

`SecurityException - if the specified security context is not an instance of AccessControlContext (e.g., is null), or is denied access to the resource specified by the given permission.`

`NullPointerException - if the permission argument is null.`

Since:

1.2

See Also:

`getSecurityContext()`, `AccessControlContext.checkPermission(java.security.Permission)`

checkCreateClassLoader

```
public void checkCreateClassLoader()
```

Throws a `SecurityException` if the calling thread is not allowed to create a new class loader.

This method calls `checkPermission` with the `RuntimePermission("createClassLoader")` permission.

If you override this method, then you should make a call to `super.checkCreateClassLoader` at the point the overridden method would normally throw an exception.

Throws:

`SecurityException - if the calling thread does not have permission to create a new class loader.`

See Also:

`ClassLoader.ClassLoader()`, `checkPermission`

checkAccess

```
public void checkAccess(Thread t)
```

Throws a `SecurityException` if the calling thread is not allowed to modify the thread argument.

This method is invoked for the current security manager by the `stop`, `suspend`, `resume`, `setPriority`, `setName`, and `setDaemon` methods of class `Thread`.

If the thread argument is a system thread (belongs to the thread group with a `null` parent) then this method calls `checkPermission` with the `RuntimePermission("modifyThread")` permission. If the thread argument is *not* a system thread, this method just returns silently.

Applications that want a stricter policy should override this method. If this method is overridden, the method that overrides it should additionally check to see if the calling thread has the `RuntimePermission("modifyThread")` permission, and if so, return silently. This is to ensure that code granted that permission (such as the JDK itself) is allowed to manipulate any thread.

If this method is overridden, then `super.checkAccess` should be called by the first statement in the overridden method, or the equivalent security check should be placed in the overridden method.

Parameters:

`t - the thread to be checked.`

Throws:

`SecurityException - if the calling thread does not have permission to modify the thread.`

`NullPointerException - if the thread argument is null.`

See Also:

`resume`, `setDaemon`, `setName`, `setPriority`, `stop`, `suspend`, `checkPermission`

checkAccess

```
public void checkAccess(ThreadGroup g)
```

Throws a `SecurityException` if the calling thread is not allowed to modify the thread group argument.

This method is invoked for the current security manager when a new child thread or child thread group is created, and by the `setDaemon`, `setMaxPriority`, `stop`, `suspend`, `resume`, and `destroy` methods of class `ThreadGroup`.

If the thread group argument is the system thread group (has a `null` parent) then this method calls `checkPermission` with the `RuntimePermission("modifyThreadGroup")` permission. If the thread group argument is *not* the system thread group, this method just returns silently.

Applications that want a stricter policy should override this method. If this method is overridden, the method that overrides it should additionally check to see if the calling thread has the `RuntimePermission("modifyThreadGroup")` permission, and if so, return silently. This is to ensure that code granted that permission (such as the JDK itself) is allowed to manipulate any thread.

If this method is overridden, then `super.checkAccess` should be called by the first statement in the overridden method, or the equivalent security check should be placed in the overridden method.

Parameters:

`g - the thread group to be checked.`

Throws:

`SecurityException - if the calling thread does not have permission to modify the thread group.`

`NullPointerException - if the thread group argument is null.`

See Also:

`destroy`, `resume`, `setDaemon`, `setMaxPriority`, `stop`, `suspend`, `checkPermission`

checkExit

```
public void checkExit(int status)
```

Throws a `SecurityException` if the calling thread is not allowed to cause the Java Virtual Machine to halt with the specified status code.

This method is invoked for the current security manager by the `exit` method of class `Runtime`. A status of `0` indicates success; other values indicate various errors.

This method calls `checkPermission` with the `RuntimePermission("exitVM."+status)` permission.

If you override this method, then you should make a call to `super.checkExit` at the point the overridden method would normally throw an exception.

Parameters:

`status - the exit status.`

Throws:

`SecurityException - if the calling thread does not have permission to halt the Java Virtual Machine with the specified status.`

See Also:

`exit, checkPermission`

checkExec

```
public void checkExec(String cmd)
```

Throws a `SecurityException` if the calling thread is not allowed to create a subprocess.

This method is invoked for the current security manager by the `exec` methods of class `Runtime`.

This method calls `checkPermission` with the `FilePermission(cmd,"execute")` permission if cmd is an absolute path, otherwise it calls `checkPermission` with `FilePermission("<<ALL FILES>>","execute")`.

If you override this method, then you should make a call to `super.checkExec` at the point the overridden method would normally throw an exception.

Parameters:

`cmd - the specified system command.`

Throws:

`SecurityException - if the calling thread does not have permission to create a subprocess.`

`NullPointerException - if the cmd argument is null.`

See Also:

`Runtime.exec(java.lang.String), Runtime.exec(java.lang.String, java.lang.String[]), Runtime.exec(java.lang.String[]), Runtime.exec(java.lang.String[], java.lang.String[]), checkPermission`

checkLink

```
public void checkLink(String lib)
```

Throws a `SecurityException` if the calling thread is not allowed to dynamic link the library code specified by the string argument file. The argument is either a simple library name or a complete filename.

This method is invoked for the current security manager by methods `load` and `loadLibrary` of class `Runtime`.

This method calls `checkPermission` with the `RuntimePermission("loadLibrary."+lib)` permission.

If you override this method, then you should make a call to `super.checkLink` at the point the overridden method would normally throw an exception.

Parameters:

lib - the name of the library.

Throws:

SecurityException - if the calling thread does not have permission to dynamically link the library.

NullPointerException - if the lib argument is null.

See Also:

Runtime.load(java.lang.String), Runtime.loadLibrary(java.lang.String), checkPermission

checkRead

public void checkRead(FileDescriptor fd)

Throws a SecurityException if the calling thread is not allowed to read from the specified file descriptor.

This method calls checkPermission with the RuntimePermission("readFileDescriptor") permission.

If you override this method, then you should make a call to super.checkRead at the point the overridden method would normally throw an exception.

Parameters:

fd - the system-dependent file descriptor.

Throws:

SecurityException - if the calling thread does not have permission to access the specified file descriptor.

NullPointerException - if the file descriptor argument is null.

See Also:

FileDescriptor, checkPermission

checkRead

public void checkRead(String file)

Throws a SecurityException if the calling thread is not allowed to read the file specified by the string argument.

This method calls checkPermission with the FilePermission(file,"read") permission.

If you override this method, then you should make a call to super.checkRead at the point the overridden method would normally throw an exception.

Parameters:

file - the system-dependent file name.

Throws:

SecurityException - if the calling thread does not have permission to access the specified file.

NullPointerException - if the file argument is null.

See Also:

checkPermission

checkRead

```
public void checkRead(String file,
                      Object context)
```

Throws a SecurityException if the specified security context is not allowed to read the file specified by the string argument. The context must be a security context returned by a previous call to getSecurityContext.

If context is an instance of AccessControlContext then the AccessControlContext.checkPermission method will be invoked with the FilePermission(file,"read") permission.

If context is not an instance of AccessControlContext then a SecurityException is thrown.

If you override this method, then you should make a call to super.checkRead at the point the overridden method would normally throw an exception.

Parameters:

file - the system-dependent filename.

context - a system-dependent security context.

Throws:

SecurityException - if the specified security context is not an instance of AccessControlContext (e.g., is null), or does not have permission to read the specified file.

NullPointerException - if the file argument is null.

See Also:

getSecurityContext(), AccessControlContext.checkPermission(java.security.Permission)

checkWrite

```
public void checkWrite(FileDescriptor fd)
```

Throws a `SecurityException` if the calling thread is not allowed to write to the specified file descriptor.

This method calls `checkPermission` with the `RuntimePermission("writeFileDescriptor")` permission.

If you override this method, then you should make a call to `super.checkWrite` at the point the overridden method would normally throw an exception.

Parameters:

`fd - the system-dependent file descriptor.`

Throws:

`SecurityException - if the calling thread does not have permission to access the specified file descriptor.`

`NullPointerException - if the file descriptor argument is null.`

See Also:

`FileDescriptor, checkPermission`

checkWrite

```
public void checkWrite(String file)
```

Throws a `SecurityException` if the calling thread is not allowed to write to the file specified by the string argument.

This method calls `checkPermission` with the `FilePermission(file,"write")` permission.

If you override this method, then you should make a call to `super.checkWrite` at the point the overridden method would normally throw an exception.

Parameters:

`file - the system-dependent filename.`

Throws:

`SecurityException - if the calling thread does not have permission to access the specified file.`

`NullPointerException - if the file argument is null.`

See Also:

`checkPermission`

checkDelete

```
public void checkDelete(String file)
```

Throws a `SecurityException` if the calling thread is not allowed to delete the specified file.

This method is invoked for the current security manager by the `delete` method of class `File`.

This method calls `checkPermission` with the `FilePermission(file,"delete")` permission.

If you override this method, then you should make a call to `super.checkDelete` at the point the overridden method would normally throw an exception.

Parameters:

`file - the system-dependent filename.`

Throws:

`SecurityException - if the calling thread does not have permission to delete the file.`

`NullPointerException - if the file argument is null.`

See Also:

`File.delete()`, `checkPermission`

checkConnect

```
public void checkConnect(String host,
                         int port)
```

Throws a `SecurityException` if the calling thread is not allowed to open a socket connection to the specified host and port number.

A port number of `-1` indicates that the calling method is attempting to determine the IP address of the specified host name.

This method calls `checkPermission` with the `SocketPermission(host+":"+port,"connect")` permission if the port is not equal to -1. If the port is equal to -1, then it calls `checkPermission` with the `SocketPermission(host,"resolve")` permission.

If you override this method, then you should make a call to `super.checkConnect` at the point the overridden method would normally throw an exception.

Parameters:

`host - the host name port to connect to.`

port - the protocol port to connect to.

Throws:

SecurityException - if the calling thread does not have permission to open a socket connection to the specified host and port.

NullPointerException - if the host argument is null.

See Also:

checkPermission

checkConnect

```
public void checkConnect(String host,
                         int port,
                         Object context)
```

Throws a SecurityException if the specified security context is not allowed to open a socket connection to the specified host and port number.

A port number of -1 indicates that the calling method is attempting to determine the IP address of the specified host name.

If context is not an instance of AccessControlContext then a SecurityException is thrown.

Otherwise, the port number is checked. If it is not equal to -1, the context's checkPermission method is called with a SocketPermission(host+":"+port,"connect") permission. If the port is equal to -1, then the context's checkPermission method is called with a SocketPermission(host,"resolve") permission.

If you override this method, then you should make a call to super.checkConnect at the point the overridden method would normally throw an exception.

Parameters:

host - the host name port to connect to.

port - the protocol port to connect to.

context - a system-dependent security context.

Throws:

SecurityException - if the specified security context is not an instance of AccessControlContext (e.g., is null), or does not have permission to open a socket connection to the specified host and port.

NullPointerException - if the host argument is null.

See Also:

getSecurityContext(),
AccessControlContext.checkPermission(java.security.Permission)

checkListen

```
public void checkListen(int port)
```

Throws a SecurityException if the calling thread is not allowed to wait for a connection request on the specified local port number.

This method calls checkPermission with the SocketPermission("localhost:"+port,"listen").

If you override this method, then you should make a call to super.checkListen at the point the overridden method would normally throw an exception.

Parameters:

port - the local port.

Throws:

SecurityException - if the calling thread does not have permission to listen on the specified port.

See Also:

checkPermission

checkAccept

```
public void checkAccept(String host,
                        int port)
```

Throws a SecurityException if the calling thread is not permitted to accept a socket connection from the specified host and port number.

This method is invoked for the current security manager by the accept method of class ServerSocket.

This method calls checkPermission with the SocketPermission(host+":"+port,"accept") permission.

If you override this method, then you should make a call to super.checkAccept at the point the overridden method would normally throw an exception.

Parameters:

host - the host name of the socket connection.

port - the port number of the socket connection.

Throws:

SecurityException - if the calling thread does not have permission to accept the connection.

NullPointerException - if the host argument is null.

See Also:

ServerSocket.accept(), checkPermission

checkMulticast

```
public void checkMulticast(InetAddress maddr)
```

Throws a SecurityException if the calling thread is not allowed to use (join/leave/send/receive) IP multicast.

This method calls checkPermission with the java.net.SocketPermission(maddr.getHostAddress(), "accept,connect") permission.

If you override this method, then you should make a call to super.checkMulticast at the point the overridden method would normally throw an exception.

Parameters:

maddr - Internet group address to be used.

Throws:

SecurityException - if the calling thread is not allowed to use (join/leave/send/receive) IP multicast.

NullPointerException - if the address argument is null.

Since:

JDK1.1

See Also:

checkPermission

checkMulticast

```
@Deprecated
public void checkMulticast(InetAddress maddr,
                           byte ttl)
```

Deprecated. *Use #checkPermission(java.security.Permission) instead*

Throws a SecurityException if the calling thread is not allowed to use (join/leave/send/receive) IP multicast.

This method calls checkPermission with the java.net.SocketPermission(maddr.getHostAddress(), "accept,connect") permission.

If you override this method, then you should make a call to super.checkMulticast at the point the overridden method would normally throw an exception.

Parameters:

maddr - Internet group address to be used.

ttl - value in use, if it is multicast send. Note: this particular implementation does not use the ttl parameter.

Throws:

SecurityException - if the calling thread is not allowed to use (join/leave/send/receive) IP multicast.

NullPointerException - if the address argument is null.

Since:

JDK1.1

See Also:

checkPermission

checkPropertiesAccess

```
public void checkPropertiesAccess()
```

Throws a SecurityException if the calling thread is not allowed to access or modify the system properties.

This method is used by the getProperties and setProperties methods of class System.

This method calls checkPermission with the PropertyPermission("*", "read,write") permission.

If you override this method, then you should make a call to super.checkPropertiesAccess at the point the overridden method would normally throw an exception.

Throws:

SecurityException - if the calling thread does not have permission to access or modify the system properties.

See Also:

System.getProperties(), System.setProperties(java.util.Properties), checkPermission

checkPropertyAccess

```
public void checkPropertyAccess(String key)
```

Throws a `SecurityException` if the calling thread is not allowed to access the system property with the specified `key` name.

This method is used by the `getProperty` method of class `System`.

This method calls `checkPermission` with the `PropertyPermission(key, "read")` permission.

If you override this method, then you should make a call to `super.checkPropertyAccess` at the point the overridden method would normally throw an exception.

Parameters:

`key - a system property key.`

Throws:

`SecurityException - if the calling thread does not have permission to access the specified system property.`

`NullPointerException - if the key argument is null.`

`IllegalArgumentException - if key is empty.`

See Also:

`System.getProperty(java.lang.String), checkPermission`

checkTopLevelWindow

```
@Deprecated
public boolean checkTopLevelWindow(Object window)
```

Deprecated. *The dependency on `AWTPermission` creates an impediment to future modularization of the Java platform. Users of this method should instead invoke `checkPermission(java.security.Permission)` directly. This method will be changed in a future release to check the permission `java.security.AllPermission`.*

Returns `false` if the calling thread is not trusted to bring up the top-level window indicated by the `window` argument. In this case, the caller can still decide to show the window, but the window should include some sort of visual warning. If the method returns `true`, then the window can be shown without any special restrictions.

See class `Window` for more information on trusted and untrusted windows.

This method calls `checkPermission` with the `AWTPermission("showWindowWithoutWarningBanner")` permission, and

returns `true` if a SecurityException is not thrown, otherwise it returns `false`. In the case of subset Profiles of Java SE that do not include the `java.awt` package, `checkPermission` is instead called to check the permission `java.security.AllPermission`.

If you override this method, then you should make a call to `super.checkTopLevelWindow` at the point the overridden method would normally return `false`, and the value of `super.checkTopLevelWindow` should be returned.

Parameters:

`window - the new window that is being created.`

Returns:

`true if the calling thread is trusted to put up top-level windows; false otherwise.`

Throws:

`NullPointerException - if the window argument is null.`

See Also:

`Window, checkPermission`

checkPrintJobAccess

```
public void checkPrintJobAccess()
```

Throws a `SecurityException` if the calling thread is not allowed to initiate a print job request.

This method calls `checkPermission` with the `RuntimePermission("queuePrintJob")` permission.

If you override this method, then you should make a call to `super.checkPrintJobAccess` at the point the overridden method would normally throw an exception.

Throws:

`SecurityException - if the calling thread does not have permission to initiate a print job request.`

Since:

`JDK1.1`

See Also:

`checkPermission`

checkSystemClipboardAccess

```
@Deprecated
public void checkSystemClipboardAccess()
```

Deprecated. *The dependency on `AWTPermission` creates an impediment to future modularization of the Java platform. Users of this method should instead invoke `checkPermission(java.security.Permission)` directly. This method will be changed in a future release to check the permission `java.security.AllPermission`.*

Throws a `SecurityException` if the calling thread is not allowed to access the system clipboard.

This method calls `checkPermission` with the `AWTPermission("accessClipboard")` permission. In the case of subset Profiles of Java SE that do not include the `java.awt` package, `checkPermission` is instead called to check the permission `java.security.AllPermission`.

If you override this method, then you should make a call to `super.checkSystemClipboardAccess` at the point the overridden method would normally throw an exception.

Throws:

`SecurityException - if the calling thread does not have permission to access the system clipboard.`

Since:

`JDK1.1`

See Also:

`checkPermission`

checkAwtEventQueueAccess

```
@Deprecated
public void checkAwtEventQueueAccess()
```

Deprecated. *The dependency on `AWTPermission` creates an impediment to future modularization of the Java platform. Users of this method should instead invoke `checkPermission(java.security.Permission)` directly. This method will be changed in a future release to check the permission `java.security.AllPermission`.*

Throws a `SecurityException` if the calling thread is not allowed to access the AWT event queue.

This method calls `checkPermission` with the `AWTPermission("accessEventQueue")` permission. In the case of subset Profiles of Java SE that do not include the `java.awt` package,

checkPermission is instead called to check the permission java.security.AllPermission.

If you override this method, then you should make a call to super.checkAwtEventQueueAccess at the point the overridden method would normally throw an exception.

Throws:

SecurityException - if the calling thread does not have permission to access the AWT event queue.

Since:

JDK1.1

See Also:

checkPermission

checkPackageAccess

```
public void checkPackageAccess(String pkg)
```

Throws a SecurityException if the calling thread is not allowed to access the package specified by the argument.

This method is used by the loadClass method of class loaders.

This method first gets a list of restricted packages by obtaining a comma-separated list from a call to java.security.Security.getProperty("package.access"), and checks to see if pkg starts with or equals any of the restricted packages. If it does, then checkPermission gets called with the RuntimePermission("accessClassInPackage."+pkg) permission.

If this method is overridden, then super.checkPackageAccess should be called as the first line in the overridden method.

Parameters:

pkg - the package name.

Throws:

SecurityException - if the calling thread does not have permission to access the specified package.

NullPointerException - if the package name argument is null.

See Also:

loadClass, getProperty, checkPermission

checkPackageDefinition

```
public void checkPackageDefinition(String pkg)
```

Throws a `SecurityException` if the calling thread is not allowed to define classes in the package specified by the argument.

This method is used by the `loadClass` method of some class loaders.

This method first gets a list of restricted packages by obtaining a comma-separated list from a call to `java.security.Security.getProperty("package.definition")`, and checks to see if `pkg` starts with or equals any of the restricted packages. If it does, then `checkPermission` gets called with the `RuntimePermission("defineClassInPackage."+pkg)` permission.

If this method is overridden, then `super.checkPackageDefinition` should be called as the first line in the overridden method.

Parameters:

`pkg - the package name.`

Throws:

`SecurityException - if the calling thread does not have permission to define classes in the specified package.`

See Also:

`ClassLoader.loadClass(java.lang.String, boolean)`, `getProperty`, `checkPermission`

checkSetFactory

```
public void checkSetFactory()
```

Throws a `SecurityException` if the calling thread is not allowed to set the socket factory used by `ServerSocket` or `Socket`, or the stream handler factory used by `URL`.

This method calls `checkPermission` with the `RuntimePermission("setFactory")` permission.

If you override this method, then you should make a call to `super.checkSetFactory` at the point the overridden method would normally throw an exception.

Throws:

`SecurityException - if the calling thread does not have permission to specify a socket factory or a stream handler factory.`

See Also:

setSocketFactory, setSocketImplFactory, setURLStreamHandlerFactory, checkPermission

checkMemberAccess

```
@Deprecated
public void checkMemberAccess(Class<?> clazz,
                              int which)
```

Deprecated. *This method relies on the caller being at a stack depth of 4 which is error-prone and cannot be enforced by the runtime. Users of this method should instead invoke checkPermission(java.security.Permission) directly. This method will be changed in a future release to check the permission java.security.AllPermission.*

Throws a SecurityException if the calling thread is not allowed to access members.

The default policy is to allow access to PUBLIC members, as well as access to classes that have the same class loader as the caller. In all other cases, this method calls checkPermission with the RuntimePermission("accessDeclaredMembers") permission.

If this method is overridden, then a call to super.checkMemberAccess cannot be made, as the default implementation of checkMemberAccess relies on the code being checked being at a stack depth of 4.

Parameters:

clazz - the class that reflection is to be performed on.

which - type of access, PUBLIC or DECLARED.

Throws:

SecurityException - if the caller does not have permission to access members.

NullPointerException - if the clazz argument is null.

Since:

JDK1.1

See Also:

Member, checkPermission

checkSecurityAccess

```
public void checkSecurityAccess(String target)
```

Determines whether the permission with the specified permission target name should be granted or denied.

If the requested permission is allowed, this method returns quietly. If denied, a SecurityException is raised.

This method creates a `SecurityPermission` object for the given permission target name and calls `checkPermission` with it.

See the documentation for `SecurityPermission` for a list of possible permission target names.

If you override this method, then you should make a call to `super.checkSecurityAccess` at the point the overridden method would normally throw an exception.

Parameters:

`target - the target name of the SecurityPermission.`

Throws:

`SecurityException - if the calling thread does not have permission for the requested access.`

`NullPointerException - if target is null.`

`IllegalArgumentException - if target is empty.`

Since:

`JDK1.1`

See Also:

`checkPermission`

getThreadGroup

```
public ThreadGroup getThreadGroup()
```

Returns the thread group into which to instantiate any new thread being created at the time this is being called. By default, it returns the thread group of the current thread. This should be overridden by a specific security manager to return the appropriate thread group.

Returns:

`ThreadGroup that new threads are instantiated into`

Since:

`JDK1.1`

See Also:

`ThreadGroup`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.